

Gebze Technical University

CSE344 SYSTEM PROGRAMMING

-

HW5 REPORT

Veysel Cemaloğlu

200104004107

Compiling and Running

Compile the code:

```
make
```

Run the program:

```
./MWCp <buffer_size> <num_customers> <source_dir> <destination_dir>
```

Clean the object files:

```
make clean
```

Problem Overview

The task at hand is to create a multithreaded C program that efficiently copies files from a source directory to a destination directory. This program must handle various types of files (regular files, directories, and FIFOs) and must do so in a way that maximizes throughput while ensuring thread safety. Additionally, the program must be robust enough to handle signals (such as SIGINT and SIGTSTP) and terminate gracefully.

In assignment 5, we introduced the use of barriers to further enhance synchronization between threads. A barrier is a synchronization primitive that ensures multiple threads wait at a certain point in the program until all threads have reached that point. This is particularly useful in scenarios where a certain phase of computation must be completed before any thread can proceed to the next phase.

To achieve these goals, the program utilizes several synchronization mechanisms, including mutexes, condition variables, and barriers:

- **Mutexes and Condition Variables:** These are used to manage shared resources and ensure synchronization between threads, preventing data races and ensuring safe access to shared data structures like the circular buffer used for file descriptors.
- **Barriers:** Introduced in assignment 5, barriers are used to synchronize the start of the file copying process. This ensures that all consumer (worker) threads begin processing files simultaneously, enhancing load balancing and overall efficiency.

By combining these synchronization primitives, the program can efficiently manage multiple threads, handle various file types, and respond appropriately to system signals, ensuring a robust and high-performance file copying operation.

PROBLEM SOLVING APPROACH

1. Checking the Given Paths

Before initiating any file operations, the program first verifies the validity of the source and destination paths. This is crucial to avoid potential issues such as attempting to copy files into a non-existent directory or copying a directory into itself.

- **Absolute Path Resolution:** The `realpath` function is used to resolve the absolute paths of the source and destination directories. This ensures that the program operates on the correct directories regardless of the relative paths provided by the user.
- **Parent Directory Check:** The program checks if the destination directory exists. If it does not, the program attempts to create the necessary parent directories.
- **Self-Copy Prevention:** The program checks if the destination directory is a subdirectory of the source directory. Copying a directory into itself would result in infinite recursion, so this situation is detected and handled appropriately.

2. Declaration of the Buffer

A circular buffer (queue) is used to store file descriptors that need to be processed (copied). This buffer serves as the communication channel between the producer (manager) thread (which enqueues file descriptors) and consumer (worker) threads (which dequeue file descriptors and perform the actual file copying).

- **Buffer Structure:** The buffer is implemented using a structure that maintains an array of file descriptors, along with indices to keep track of the front and rear of the queue. This allows for efficient enqueue and dequeue operations.
- **Buffer Capacity:** The buffer's capacity is specified by the user and dictates how many file descriptors can be stored in the buffer at any given time.

3. Creating Threads

The program uses one producer (manager) thread and multiple consumer (worker) threads to achieve parallelism.

- **Producer (Manager) Thread:** Responsible for traversing the source directory, opening files, and enqueueing file descriptors into the buffer.
- **Consumer (Worker) Threads:** Responsible for dequeuing file descriptors from the buffer and performing the actual file copy operations.

4. Mutexes and Condition Variables

To ensure thread safety and synchronization, mutexes and condition variables are used extensively.

- **Mutexes:** Protect shared resources such as the buffer, standard output, and file descriptor counters.
- **Condition Variables:** Facilitate communication between threads, allowing them to wait for certain conditions to be met (e.g., buffer not being full for the producer, buffer not being empty for the consumers).

5. Implementation of Barriers

In the context of our file copying program, the barrier was used to synchronize the start of the file copying process. This ensures that all consumer (worker) threads start processing files at the same time, thereby improving the overall efficiency and load balancing of the program.

Barrier Initialization:

Before creating the threads, we initialize a barrier that will be used to synchronize their start. The barrier is initialized with a count equal to the number of consumer threads plus one (for the producer thread):

```
pthread_barrier_t barrier;
```

```
pthread_barrier_init(&barrier, NULL, num_customers + 1);
```

Barrier Usage in Threads:

- **Producer Thread:** The producer thread waits at the barrier after initializing the buffer and before starting to enqueue file descriptors. This ensures that the producer does not start enqueueing before all consumer threads are ready to dequeue.

```
pthread_barrier_wait(&barrier);
```

- **Consumer Threads:** Each consumer thread waits at the barrier after being created and before starting to dequeue file descriptors. This ensures that no consumer thread starts processing before the producer is ready.

```
pthread_barrier_wait(&barrier);
```

Barrier Destruction:

After all threads have synchronized and started processing, the barrier is no longer needed and can be destroyed:

```
pthread_barrier_destroy(&barrier);
```

6. Producer (Manager) Thread

The producer thread performs the following tasks:

1. **Directory Traversal:** Recursively traverses the source directory.
2. **File Descriptor Management:** Opens source and destination files, then enqueues their file descriptors into the buffer.
3. **Directory Creation:** Creates corresponding directories in the destination path.
4. **FIFO Handling:** Handles FIFO files by creating them in the destination directory.

7. Consumer (Worker) Threads

The consumer threads perform the following tasks:

1. **Buffer Dequeue:** Dequeues file descriptors from the buffer.
2. **File Copy:** Reads from the source file descriptor and writes to the destination file descriptor in chunks, ensuring that all data is copied correctly.
3. **Resource Management:** Closes file descriptors and frees allocated memory once the copy operation is complete.

8. Handle Signals

The program includes signal handlers to manage termination signals (SIGINT and SIGTSTP).

- **Signal Handling:** When a termination signal is received, the signal handler sets a flag to indicate that the program should terminate. This flag is checked by both producer and consumer threads to ensure they exit gracefully.
- **Broadcasting Condition Variables:** The signal handler broadcasts condition variables to wake up any waiting threads, allowing them to exit.

9. Terminating Threads

Graceful termination of threads is essential to ensure that all resources are properly released and no data is lost.

- **Thread Joining:** The main function waits for all threads to complete their execution using `pthread_join`.
- **Resource Cleanup:** Mutexes and condition variables are destroyed, and dynamically allocated memory is freed.

- **Statistics Printing:** Upon completion, the program prints statistics such as the number of files copied, the total bytes copied, and the total time taken for the operation.

TEST CASES:

Test1: valgrind ./MWCp 10 10 ../testdir/src/libvterm ../tocopy :

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/Desktop/systemprog/HW5/hw4test/put_your_codes_h...
|# fileio.c                200176 bytes copied successfully #
|# config.mk.dist          95 bytes copied successfully #
|# gui_beval.c             31823 bytes copied successfully #
|# ex_cmdidxs.h            4096 bytes copied successfully #
|# autocmd.c              66845 bytes copied successfully #
|# ex_eval.c              68682 bytes copied successfully #
|# .gitignore              130 bytes copied successfully #
|# gui_xmebw.c            39605 bytes copied successfully #
|# getchar.c             129606 bytes copied successfully #
|# gui_at_sb.c            33157 bytes copied successfully #
|# hangulin.c            61168 bytes copied successfully #
|# change.c              59983 bytes copied successfully #
|# ex_cmds.c             196359 bytes copied successfully #
|# MRT10.pdf             4033747 bytes copied successfully #
|# isl1551098739430.pdf   14752648 bytes copied successfully #

-----STATISTICS-----
Consumers: 10 - Buffer Size: 10
Number of Regular Files: 194
Number of FIFO Files: 0
Number of Directories: 7
TOTAL BYTES COPIED: 25009680
TOTAL TIME: 00:00.713 (min:sec.milli)
==6154==
==6154== HEAP SUMMARY:
==6154==    in use at exit: 0 bytes in 0 blocks
==6154==   total heap usage: 217 allocs, 217 frees, 1,062,904 bytes allocated
==6154==
==6154== All heap blocks were freed -- no leaks are possible
==6154==
==6154== For lists of detected and suppressed errors, rerun with: -s
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/HW5/hw4test/put_your_codes_here$
```

Test2: ./MWCp 10 4 ../testdir/src/libvterm/src ../tocopy :

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/Desktop/systemprog/HW5/hw4test/put_your_codes_h...
|# dehqx.py                1036 bytes copied successfully #
|# blob.c                  4880 bytes copied successfully #
|# if_py_both.h            151933 bytes copied successfully #
|# configure                318 bytes copied successfully #
|# utf8.h                  1217 bytes copied successfully #
|# feature.h               32609 bytes copied successfully #
|# getchar.c               129606 bytes copied successfully #
|# iscygpty.h              1527 bytes copied successfully #
|# autocmd.c               66845 bytes copied successfully #
|# config.h.in             11717 bytes copied successfully #
|# gui_x11_pm.h            2405 bytes copied successfully #
|# gui_beval.c             31823 bytes copied successfully #
|# gui_at_sb.h             5934 bytes copied successfully #
|# ex_eval.c               68682 bytes copied successfully #
|# config.mk.dist          95 bytes copied successfully #
|# hangulin.c              61168 bytes copied successfully #
|# ex_cmdidxs.h            4096 bytes copied successfully #
|# gui_at_sb.c             33157 bytes copied successfully #
|# gui_xmebw.c             39605 bytes copied successfully #
|# ex_cmds.c               196359 bytes copied successfully #
|# change.c                59983 bytes copied successfully #
|# MRT10.pdf               4033747 bytes copied successfully #
|# isl1551098739430.pdf    14752648 bytes copied successfully #

-----STATISTICS-----
Consumers: 4 - Buffer Size: 10
Number of Regular Files: 140
Number of FIFO Files: 0
Number of Directories: 2
TOTAL BYTES COPIED: 24873082
TOTAL TIME: 00:00.097 (min:sec.milli)
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/HW5/hw4test/put_your_codes_here$
```


Test3: ./MWCp 10 100 ../testdir ../toCopy :

```
veysel@veysel-Yoga-Slim-7-14ARE05: ~/Desktop/systemprog/HW5/hw4test/put_your_codes_h...
|# README_bindos.txt          701 bytes copied successfully #
|# README_srcdos.txt          487 bytes copied successfully #
|# runtime.info               624 bytes copied successfully #
|# README_src.txt              346 bytes copied successfully #
|# README_amisrc.txt           389 bytes copied successfully #
|# README_unix.txt             365 bytes copied successfully #
|# README_w32s.txt             612 bytes copied successfully #
|# Xxd.info                    835 bytes copied successfully #
|# README_dos.txt              6306 bytes copied successfully #
|# README.txt.info             582 bytes copied successfully #
|# src.info                    624 bytes copied successfully #
|# README_ami.txt.info         582 bytes copied successfully #
|# README_extra.txt            1453 bytes copied successfully #
|# README_ami.txt              1252 bytes copied successfully #
|# Contents                    1019 bytes copied successfully #
|# Contents.info               582 bytes copied successfully #
|# # ../toCopy/.github          directory created #
|# README_amisrc.txt.info      582 bytes copied successfully #
|# README_os2.txt              234 bytes copied successfully #
|# # ../toCopy/.github/ISSUE_TEMPLATE directory created #
|# feature_request.md           806 bytes copied successfully #
|# bug_report.md               1072 bytes copied successfully #
|# README.md                    5946 bytes copied successfully #

-----STATISTICS-----
Consumers: 100 - Buffer Size: 10
Number of Regular Files: 3116
Number of FIFO Files: 0
Number of Directories: 151
TOTAL BYTES COPIED: 73520554
TOTAL TIME: 00:00.240 (min:sec.milli)
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/HW5/hw4test/put_your_codes_here$
```

CONCLUSION

By using a combination of multithreading, synchronization primitives, and signal handling, the program efficiently copies files from a source directory to a destination directory. The use of a buffer for file descriptors allows for smooth communication between producer (manager) and consumer (worker) threads, while mutexes, condition variables and barrier ensure thread safety and coordination. This approach leverages the strengths of multithreading to achieve high throughput and robustness in file copy operations.